

AST design

Compiler implementation

Ivan Trepakov

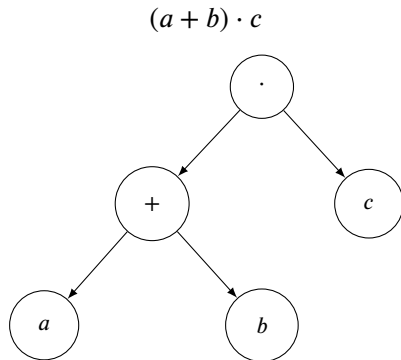
NSU Sys.Pro

Abstract syntax trees

Grammar and tree

$$e ::= n \mid x \mid e_1 + e_2 \mid e_1 \cdot e_2$$

- A parse tree records every derivation step
- An **abstract** syntax tree keeps only the structure
 - Parentheses, precedence, spelling of keywords: gone
- The tree shape mirrors the grammar productions
- Every later stage of the compiler works on this tree
 - Semantic checks, transformations, IR generation



Representations

Tagged union

```
enum expr_kind {  
    EXPR_INT,  
    EXPR_VAR,  
    EXPR_ADD,  
    EXPR_MUL,  
};  
  
struct expr {  
    enum expr_kind kind;  
    int64_t int_val;  
    const char *var_name;  
    struct expr *l, *r;  
};
```

arena-allocated

Class hierarchy

```
sealed trait Expr  
case class IntLit(value: Long)  
    extends Expr  
case class Var(name: String)  
    extends Expr  
case class Add(l: Expr, r: Expr)  
    extends Expr  
case class Mul(l: Expr, r: Expr)  
    extends Expr
```

GC or arena-allocated

Abstract Data Type (ADT)

```
enum Expr:  
    case IntLit(value: Long)  
    case Var(name: String)  
    case Add(l: Expr, r: Expr)  
    case Mul(l: Expr, r: Expr)
```

GC, immutable

Representation of passes

Internal: methods on nodes

```
sealed trait Expr:  
  def eval(env: Env): Long  
  
case class Add(l, r) extends Expr  
  def eval(env: Env): Long =  
    l.eval(env) + r.eval(env)
```

External: functions over the tree

```
def eval(e: Expr, env: Env): Long = e match  
  case IntLit(v) => v  
  case Var(x)    => env(x)  
  case Add(l, r) => eval(l, env) + eval(r, env)  
  case Mul(l, r) => eval(l, env) * eval(r, env)
```


Switch

- Dispatch on the **tag field**
- No exhaustiveness checks
- State needs explicit params

```
int64_t eval(const struct expr *e) {  
    switch (e->kind) {  
        case EXPR_INT:  
            return e->int_val;  
        case EXPR_VAR:  
            return lookup(name);  
        case EXPR_ADD:  
            return eval(e->l) + eval(e->r);  
        case EXPR_MUL:  
            return eval(e->l) * eval(e->r);  
    }  
}
```

Switch

- Dispatch on the **tag field**
- No exhaustiveness checks
- State needs explicit params

Match

- Supported in many modern languages
- Exhaustiveness checks
- State needs explicit params

```
sealed trait Expr
case class IntLit(value: Long) extends Expr
case class Var(name: String) extends Expr
case class Add(l: Expr, r: Expr) extends Expr
case class Mul(l: Expr, r: Expr) extends Expr
```

```
def eval(e: Expr): Long = e match
  case IntLit(v) => v
  case Var(x)    => env(x)
  case Add(l, r) => eval(l) + eval(r)
  case Mul(l, r) => eval(l) * eval(r)
```

External traversal

Switch

- Dispatch on the **tag field**
- No exhaustiveness checks
- State needs explicit params

Match

- Supported in many modern languages
- Exhaustiveness checks
- State needs explicit params

Visitor

- Object-oriented alternative
- No exhaustiveness checks
- Visitor carries state

```
trait Visitor[R] {  
  def visitInt(e: IntLit): R  
  def visitVar(e: Var): R  
  def visitAdd(e: Add): R  
  def visitMul(e: Mul): R  
}  
  
case class Add(l: Expr, r: Expr) extends Expr:  
  def accept(v: Visitor[R]): R =  
    v.visitAdd(this)
```

Internal vs external

Trade-offs

- **Internal**: one new node kind is one new class, but the pass logic is scattered across all of them
- **External**: one new pass is one new function/class, but adding a node kind means touching every pass

Good for frontend: compilers run *many passes* over a *slowly growing* node set. External traversal pays off

	Add pass	Add node
Internal	painful	easy
External	easy	painful

Visitor: fields on the object

```
class EmitVisitor(  
  val out: StringBuilder,  
  var scopes: SymbolTable,  
  var loops: List[LoopCtx]  
) extends Visitor[Value]:  
  
  def visitAdd(e: Add): Value =  
    builder.add(  
      e.l.accept(this),  
      e.r.accept(this))
```

Match/switch: explicit parameters

```
def emit(e: Expr)(  
  out: StringBuilder,  
  scopes: SymbolTable,  
  loops: List[LoopCtx]  
): Value = e match  
  
  case Add(l, r) =>  
    emit(l)(out, scopes, loops)  
    emit(r)(out, scopes, loops)
```

Tagged union

- GCC: `tree_code` over `union tree_node`
- CPython: `op` enums over `node structs` (classic form)
- Go: `ir.0p` over a `node struct`
- Zig: `std.zig.Ast`, tag per node

Class hierarchy

- scalac: `Trees.scala`, `Tree` subclasses
- Clang: `Stmt/Expr` hierarchy
- javac: `JCTree` subclasses
- V8: `AstNode` subclasses

ADT

- Scala 3 (dotty): the same `Tree` as `case class ADT`
- OCaml compiler: `Parsetree`
- F#: `SynExpr`
- Elm: `Expr`

Traversal is always external: `switch`, `visitor`, or `match`

class hierarchy + arena

- Traversal: StmtVisitor, the visitor pattern over a class hierarchy, state in the visitor
- Stmt/Expr: about a hundred node classes, one per syntactic form
- Every node carries a kind tag: cheap isa/dyn_cast without RTTI
- Uniform child storage: BinaryOperator keeps Stmt *SubExprs[2]: children typed Stmt*, not dedicated fields
- All nodes allocated in the ASTContext arena; freed once, when the translation unit is done

```
class BinaryOperator : public Expr {  
    enum { LHS, RHS, END_EXPR };  
    Stmt *SubExprs[END_EXPR];  
  
public:  
    typedef BinaryOperatorKind Opcode;  
  
    const Expr *getLHS() const {  
        return cast<Expr>(SubExprs[LHS]);  
    }  
};
```

tagged union + arena

- One node type: union `tree_node`, the tree is a pointer to it
- enum `tree_code`: over two hundred codes, one per form (`INTEGER_CST`, `PLUS_EXPR`, ...)
- Traversal: switch on the code, guided by `tree_code_class`
- Nodes live in the GC-managed arena (GGC), collected a whole generation at once
- The same tree from parser to code generation

```
/* tree.def: one line per code */
DEFTREECODE (INTEGER_CST,
             "integer_cst", tcc_constant, 0)
DEFTREECODE (PLUS_EXPR,
             "plus_expr", tcc_binary, 2)

/* tree-core.h */
enum tree_code : unsigned {
#include "all-tree.def" MAX_TREE_CODES
};

/* the node itself */
union tree_node {
    struct tree_base base;
    struct tree_int_cst int_cst;
};
```

class hierarchy, one tree per pass

- abstract class `Tree[T]`: node classes in `Trees.scala`, one per form (`Ident`, `Apply`, ...)
- `T` is the pass: `Untyped` from the parser, typed after the typer; the type lives in the node
- Traversal: `TreeTraverser` / `TreeMap` base classes, clients override per-node callbacks
- Types are set copy-on-write: a tree is reused across passes
- case class nodes double as an ADT: match works on them too

```
abstract class Tree[+T <: Untyped]  
  extends Positioned, SrcPos
```

```
case class Ident[+T <: Untyped]  
  (name: Name) extends RefTree[T]
```

```
case class Apply[+T <: Untyped]  
  (fun: Tree[T], args: List[Tree[T]])
```

```
case class Literal[+T <: Untyped]  
  (const: Constant) extends Tree[T]
```

`T` selects the pass: `Untyped` or the typed tree

IR generation

AST travelsals

- Name resolution
- type checking,
- constant folding
- IR generation

IR generation

- Expressions produce values
- Statements produce effects: `x = ...` emits a store
- `alloca` reserves a stack slot per variable; load/store move values
- No registers are assigned by us: LLVM handles register allocation

```
x = a + b;
```

```
%x = alloca i64
```

```
%a = alloca i64
```

```
%b = alloca i64
```

```
; a + b
```

```
%1 = load i64, ptr %a
```

```
%2 = load i64, ptr %b
```

```
%3 = add i64 %1, %2
```

```
; x =
```

```
store i64 %3, ptr %x
```


IR generation

Switch

- `tmp(...)` allocates the next temporary
%n and appends the line
- The recursion order is the **instruction order**: operands first, add after

```
/* returns the name of the result var */  
const char *emit(struct expr *e) {  
    switch (e->kind) {  
  
        case EXPR_VAR:  
            return tmp("load i64, ptr %%%s",  
                       e->var_name);  
  
        case EXPR_ADD: {  
            const char *l = emit(e->l);  
            const char *r = emit(e->r);  
            return tmp("add i64 %s, %s", l, r);  
        }  
    }  
}
```

IR generation

Switch

- `tmp(...)` allocates the next temporary
%n and appends the line
- The recursion order is the **instruction order**: operands first, add after

Match

- Exhaustiveness checks
- Context (output buffer, scopes, loop stack) is threaded through parameters

```
def emit(e: Expr): String = e match
```

```
  case Var(x) =>  
    line(s"load i64, ptr %$x")
```

```
  case Add(l, r) =>  
    val a = emit(l)  
    val b = emit(r)  
    line(s"add i64 $a, $b")
```

```
  case Mul(l, r) =>  
    val a = emit(l)  
    val b = emit(r)  
    line(s"mul i64 $a, $b")
```

IR generation

Switch

- `tmp(...)` allocates the next temporary
%n and appends the line
- The recursion order is the **instruction order**: operands first, add after

Match

- Exhaustiveness checks
- Context (output buffer, scopes, loop stack) is threaded through parameters

Visitor

- The output buffer, symbol table, builder, all live in the visitor object
- `accept(this)` passes the visitor down: children reuse the same context

```
class EmitVisitor(  
  val out: StringBuilder  
) extends Visitor[String]:  
  
  def visitVar(e: Var): String =  
    line(out, s"load i64, ptr %${e.name}")  
  
  def visitAdd(e: Add): String = {  
    val l = e.l.accept(this)  
    val r = e.r.accept(this)  
    line(out, s"add i64 $l, $r")  
  }
```

LLVM C API

- Our pseudo-emit writes IR text directly; the C API builds the same instructions as objects

```
/* C API */
```

```
LLVMValueRef L = LLVMBuildLoad2(  
    Builder, I64, Slot, "");  
LLVMValueRef S = LLVMBuildAdd(  
    Builder, L, R, "tmp");
```

Text

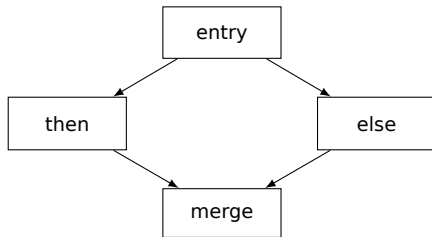
```
; the same instructions, text form  
%1 = load i64, ptr %a  
%tmp = add i64 %1, %2
```

- **Builder** = insertion point: where new instructions are appended
- %n = SSA temporaries, one per instruction result
- `alloca` / `load` / `store` / `br` map one to one

Control flow

- if splits the control flow
- The function body is a list of **basic blocks**, each ending with a **terminator**
- br is the terminator for branches: a condition and two targets
- The emitter creates the blocks, remembers which is which, fills them in order

if

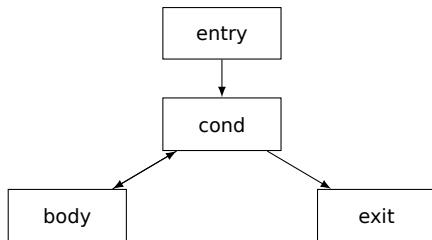


```
br i1 %cond, label %then, label %else
```

Control flow

- Same building blocks: blocks, terminators, br
- The back edge makes it a loop
- New blocks are appended; the emitter tracks the **current** block explicitly, switching to it when control flow merges

while



cond:

```
br i1 %c, label %body, label %exit
```

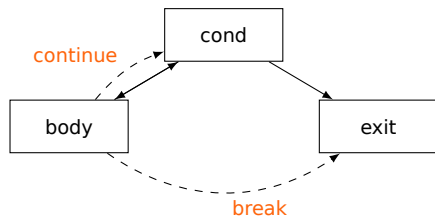
body:

```
...
```

```
br label %cond
```

break and continue

- break / continue are just jumps, but to blocks of the **enclosing loop**
- The node itself does not know its target
- The emitter keeps a stack of (break target, continue target) pairs, pushed by while / for, popped after
- Same idea as symbol tables: the traversal carries context that the AST does not contain
- Visitor: fields on the object
- Match: explicit parameters



break → exit block
continue → condition block

Q&A